code:

```
#include <SoftwareSerial.h>
#include <Servo.h>
#include <LiquidCrystal.h>

// Define software serial pins for ESP8266 communication
SoftwareSerial esp8266(10, 11); // RX, TX

// WiFi credentials - replace with your actual SSID and password
const char* ssid = "ssid"; // WiFi network name
const char* password = "pass"; //WiFi password

// Define pins for components
const int triggerPin = 2;
const int echoPin = 3;
const int redLedPin = 4;
const int greenLedPin = 5;
const int buzzerPin = 7;
const int servoPin = 9;

// LCD Pins for standard 16-pin connection
const int rs = A0;
const int en = A1;
const int d4 = A2;
const int d5 = A3;
```

```
const int d6 = A4;
const int d7 = A5;

// Constants
const int doorClosedAngle = 0;
const int doorOpenAngle = 90;
const int proximityThreshold = 3; // Changed from 30cm to 3cm as requested

// Variables
bool doorOpen = false;
unsigned long doorOpenedTime = 0;
bool autoModeEnabled = true;

// Initialize objects
Servo doorServo;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

void setup() {
  // Initialize pins
  pinMode(triggerPin, OUTPUT);
  pinMode(echoPin, INPUT);
  pinMode(redLedPin, OUTPUT);
  pinMode(greenLedPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);

  // Initialize components
  doorServo.attach(servoPin);
```

```
doorServo.write(doorClosedAngle); // Ensure door starts closed

// Initialize LED states
digitalWrite(redLedPin, HIGH); // Red means door closed
digitalWrite(greenLedPin, LOW);

// Initialize LCD
lcd.begin(16, 2);
lcd.setCursor(0, 0);
lcd.print("Smart Door");
lcd.setCursor(0, 1);
lcd.print("Door: Closed");

// Start serial communications
Serial.begin(9600);
esp8266.begin(9600);
Serial.println("Smart Door System Initializing...");

// Reset module and set up server
sendCommand("AT+RST", 2000);
delay(1000);

// Set ESP8266 baud rate
sendCommand("AT+UART_DEF=9600,8,1,0,0", 1000);
sendCommand("AT+CWMODE=1", 1000);
delay(500);
```

```
  // Connect to WiFi
  connectWiFi();

  // Setup the server
  sendCommand("AT+CIPMUX=1", 1000);
  sendCommand("AT+CIPSERVER=0", 1000); // Close any existing server
  delay(500);
  sendCommand("AT+CIPSERVER=1,80", 1000); // Start server on port 80
  Serial.println("Server started. Smart Door ready!");
}

void loop() {
  // Check for ESP8266 commands
  checkESP8266();

  // Only check proximity if auto mode is enabled
  if (autoModeEnabled) {
    checkProximity();
  }

  // Check if door needs to be closed after being open for a while
  if (doorOpen && (millis() - doorOpenedTime > 5000)) {
    closeDoor();
  }
}

void connectWiFi() {
```

```
  Serial.println("Connecting to WiFi...");
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Connecting WiFi");

  // Form the AT command with SSID and password
  String cmd = "AT+CWJAP=\"";
  cmd += ssid;
  cmd += "\",\"";
  cmd += password;
  cmd += "\"";

  // Send command to connect to WiFi
  sendCommand(cmd, 10000); // Longer timeout for WiFi connection

  // Get the IP address
  getIP();
}

void getIP() {
  // Get the IP address
  esp8266.println("AT+CIFSR");
  delay(2000);
  String response = "";
  while (esp8266.available()) {
    char c = esp8266.read();
    response += c;
```

```
  Serial.write(c); // Echo to serial monitor
}

// Parse the IP address from the response
int startIndex = response.indexOf("STAIP,\"") + 7;
int endIndex = response.indexOf("\"", startIndex);
if (startIndex > 7 && endIndex > startIndex) {
  String ip = response.substring(startIndex, endIndex);
  Serial.print("IP Address: ");
  Serial.println(ip);

  // Display IP on LCD briefly
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("IP Address:");
  lcd.setCursor(0, 1);
  lcd.print(ip);
  delay(3000);

  // Return to normal display
  updateLCDStatus();
} else {
  Serial.println("Failed to get IP address");
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("WiFi Error");
  delay(2000);
```

```
    updateLCDStatus();
  }
}

void updateLCDStatus() {
  // Update the LCD with current status
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print(autoModeEnabled ? "Mode: Auto " : "Mode: Manual ");
  lcd.setCursor(0, 1);
  lcd.print("Door: ");
  lcd.print(doorOpen ? "Open " : "Closed");
}

void checkProximity() {
  // Measure distance using ultrasonic sensor
  long duration, distance;

  digitalWrite(triggerPin, LOW);
  delayMicroseconds(2);
  digitalWrite(triggerPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(triggerPin, LOW);

  duration = pulseIn(echoPin, HIGH);
  distance = duration * 0.034 / 2;
```

```
  // Update LCD with distance
  lcd.setCursor(0, 0);
  lcd.print(autoModeEnabled ? "Mode: Auto " : "Mode: Manual ");
  lcd.print("D:");
  lcd.print(distance);
  lcd.print("cm ");

  // If someone is close and door is closed, open it
  if (distance < proximityThreshold && !doorOpen) {
    openDoor();
  }
}

void openDoor() {
  // Sound buzzer briefly
  tone(buzzerPin, 1000, 200);

  // Move servo to open position
  doorServo.write(doorOpenAngle);

  // Update door state
  doorOpen = true;
  doorOpenedTime = millis();

  // Update LEDs
  digitalWrite(redLedPin, LOW);
  digitalWrite(greenLedPin, HIGH);
```

```
  // Update LCD
  lcd.setCursor(0, 1);
  lcd.print("Door: Open  ");
}

void closeDoor() {
  // Sound buzzer briefly
  tone(buzzerPin, 500, 200);

  // Move servo to closed position
  doorServo.write(doorClosedAngle);

  // Update door state
  doorOpen = false;

  // Update LEDs
  digitalWrite(redLedPin, HIGH);
  digitalWrite(greenLedPin, LOW);

  // Update LCD
  lcd.setCursor(0, 1);
  lcd.print("Door: Closed");
}

void toggleAutoMode() {
  autoModeEnabled = !autoModeEnabled;
```

```
  // Update LCD
  lcd.setCursor(0, 0);
  if (autoModeEnabled) {
    lcd.print("Mode: Auto  ");
  } else {
    lcd.print("Mode: Manual");
  }
}

void checkESP8266() {
  if (esp8266.available()) {
    String data = "";
    unsigned long startTime = millis();

    // Read data with timeout to ensure we get complete requests
    while ((millis() - startTime) < 1000) {
      if (esp8266.available()) {
        char c = esp8266.read();
        data += c;
        if (data.indexOf("\r\n\r\n") != -1) {
          break; // End of HTTP request
        }
      }
    }

    // Look for an HTTP request
```

```
  if (data.indexOf("+IPD") >= 0) {
    // Extract connection ID
    int connectionId = data.charAt(data.indexOf("+IPD,") + 5) - '0';

    // Process different URLs
    if (data.indexOf("GET /open") >= 0) {
      openDoor();
      delay(100);
      sendResponse(connectionId);
    }
    else if (data.indexOf("GET /close") >= 0) {
      closeDoor();
      delay(100);
      sendResponse(connectionId);
    }
    else if (data.indexOf("GET /auto") >= 0) {
      toggleAutoMode();
      delay(100);
      sendResponse(connectionId);
    }
    else {
      // Default homepage
      delay(100);
      sendResponse(connectionId);
    }
  }
}
```

```
}

void sendCommand(String command, int timeout) {
  Serial.print("Sending command: ");
  Serial.println(command);
  esp8266.println(command);

  long startTime = millis();
  while ((millis() - startTime) < timeout) {
    if (esp8266.available()) {
      Serial.write(esp8266.read());
    }
  }
  Serial.println(); // Add line break after response
}

void sendResponse(int connectionId) {
  // Create a very minimal HTML page to save memory
  String html = "<html><body><h2>Smart Door</h2>";

  // Status section
  html += "<p>Door: ";
  html += doorOpen ? "OPEN" : "CLOSED";
  html += "</p>";
  html += "<p>Mode: ";
  html += autoModeEnabled ? "AUTO" : "MANUAL";
  html += "</p>";
```

```
  // Control buttons with minimal styling
  html += "<p><a href='/open'><button>OPEN</button></a> ";
  html += "<a href='/close'><button>CLOSE</button></a></p>";
  html += "<p><a href='/auto'><button>TOGGLE MODE</button></a></p>";
  html += "</body></html>";

  // Send the response length command
  String cipSend = "AT+CIPSEND=";
  cipSend += connectionId;
  cipSend += ",";
  cipSend += html.length() + 40; // Add extra bytes for HTTP headers
  sendCommand(cipSend, 500);

  // Send the HTTP response
  esp8266.print("HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n");
  esp8266.print(html);
  delay(100);

  // Close the connection
  String closeCommand = "AT+CIPCLOSE=";
  closeCommand += connectionId;
  sendCommand(closeCommand, 500);
}
```

---

connections:

ESP8266 WiFi Module

Connected to Arduino via Software Serial

RX Pin: Connected to Arduino digital pin 11

TX Pin: Connected to Arduino digital pin 10

EN:3.3v supply (not explicitly shown in code but required)

3.3V: Needs 3.3V supply (not explicitly shown in code but required)

GND: Connected to Arduino GND

Ultrasonic Sensor (HC-SR04)

Trigger Pin: Connected to Arduino digital pin 2

Echo Pin: Connected to Arduino digital pin 3

VCC: Connected to Arduino 5V (not explicitly shown but required)

GND: Connected to Arduino GND (not explicitly shown but required)

LCD Display (16x2)

Using standard 16-pin connection mode with the following pins:

RS (Register Select): Connected to Arduino analog pin A0

EN (Enable): Connected to Arduino analog pin A1

D4 (Data pin 4): Connected to Arduino analog pin A2

D5 (Data pin 5): Connected to Arduino analog pin A3

D6 (Data pin 6): Connected to Arduino analog pin A4

D7 (Data pin 7): Connected to Arduino analog pin A5

VSS: Connected to Arduino GND (not explicitly shown but required)

VDD: Connected to Arduino 5V (not explicitly shown but required)

V0: Connected to a potentiometer for contrast control (not explicitly shown but required)

RW: Typically connected to GND for write-only mode (not explicitly shown)

A: Connected to Arduino 5V via a resistor (not explicitly shown but required)

K: Connected to Arduino GND (not explicitly shown but required)

Servo Motor

Yellow/Orange(Signal): Connected to Arduino digital pin 9

Red(VCC): Connected to external power supply or Arduino 5V (not explicitly shown but required)

Black/Brown(GND): Connected to Arduino GND (not explicitly shown but required)

LEDs

Red LED: +ve leg Connected to Arduino digital pin 4 , -ve leg to GND

Green LED: +ve leg Connected to Arduino digital pin 5 , -ve leg to GND

Buzzer

Positive leg: Connected to Arduino digital pin 7

Negative leg: Connected to Arduino GND (not explicitly shown but required)

don’t forget to connect potentiometer to v5 and GND